EMERGING TECHNOLOGIES | TEAM SPEAKER NOTES

Presentation speaking order

Sơn Nguyên

English narration for assigned slides, in the classroom display order.

PRESENTER	ROLE	SLIDES	COVERAGE
Trần Việt Hải	Opening	**1–4**	Introduction, problem, project goals
Lâm Xê Khoa	Main presenter 1	**5–15**	LLMs, RAG, architecture, source PDFs, metadata
Mai Tuấn Dũng	Main presenter 2	**16–26**	Chunking, embeddings, FAISS, pipelines, LCEL, prompting
Sơn Nguyên	Main presenter 3	**27–37**	Ollama, app.py, logging, evaluation, risks, conclusion
Châu Ngọc Đức	Demo lead	**38**	Observed LangChain question; launch/operate demo
Trần Việt Hải	Demo + closing	**39–40**	Out-of-PDF question; instructor questions; closing

Display order

Slides 1–30 → original slides 33–39 → original slides 31–32 → final Questions slide 40. The presentation-order PowerPoint already reflects this sequence.

Duc operates the Windows 11 terminal and introduces the real LangChain screenshot. Hai demonstrates an out-of-PDF question, enters the instructor's questions, and closes the presentation. Both can assist with the live Q&A.

Demonstration safeguard

Use only observed outputs and verify answers against the sample PDF. A retrieved source list alone does not establish that the generated answer is supported. The old slides 31–32 become slides 38–39 in the speaking-order edition.

Ollama and Qwen2.5

WHAT TO SAY

The application's language model runs locally through Ollama. The default configuration requests qwen2.5:3b, and the setup script downloads that model. Local inference reduces the need to send application prompts to a hosted API, although the full privacy story still depends on local storage, logs, and how the environment is managed. Generation speed varies with hardware, question complexity, and available context. The two screenshot timings should not be interpreted as a reproducible benchmark.

KEY TAKEAWAY

Ollama serves the selected Qwen model on the user's machine.

Console Application Loop

WHAT TO SAY

The app.py file prints a header, confirms that at least one PDF exists, checks Ollama, and loads or builds the vector store. It then repeatedly asks for a question, calls answer_question, prints the answer and retrieved source metadata, and reports elapsed time. The loop supports several exit words, including exit, quit, q, and thoat. It also handles empty questions and common errors. The interface is a Windows console, not a web application.

KEY TAKEAWAY

The main loop coordinates all user-facing steps.

Question and Answer History

WHAT TO SAY

The save_history function writes one JSON object per line to a JSONL file. It records the timestamp, question, answer, retrieved source metadata, and measured elapsed time. This provides a basic audit trail for reviewing outputs and comparing results across questions. The structure shown here is illustrative; the numeric timing is taken from a screenshot, while actual saved lines depend on the local run. Because histories may contain sensitive questions or answers, the log folder should be protected.

KEY TAKEAWAY

Logging supports later inspection, but logs can contain sensitive data.

Code Walk-Through Summary

WHAT TO SAY

Four modules explain most of the architecture. app.py handles the console interface and logging. pipeline.py decides whether to build or reuse the index. vector_store.py handles multilingual embeddings and FAISS persistence. qa.py performs retrieval, formats the prompt context, and uses an LCEL chain for generation. When the instructor asks which file implements a feature, these are the clearest entry points for our explanation.

KEY TAKEAWAY

Module boundaries match the real execution flow.

Interpreting the Results

WHAT TO SAY

The screenshots confirm that a local terminal interaction occurred, that the program displayed source metadata, and that two individual elapsed times were reported. They also reveal an important evaluation issue. The refusal screenshot demonstrates one successful refusal, while the LangChain definition screenshot is not supported by the contents of the bundled sample PDF. We should not present these two timings as a speed comparison or treat the retrieved list as proof of answer correctness. Broader claims require controlled testing.

KEY TAKEAWAY

Observe carefully; do not turn two screenshots into a benchmark.

Observed Response Times

WHAT TO SAY

Our first terminal screenshot reports 9.52 seconds for the LangChain question, while the second reports 2.10 seconds for the unsupported question. These are real observations from the supplied images, not controlled benchmark averages. The questions are different, and the screenshot set does not tell us the hardware load, exact context length, or repeated-run variance. A fair latency comparison would repeat matched question sets under the same conditions and report a distribution of times.

KEY TAKEAWAY

Two single-run timings cannot establish system speed.

What the Test Suite Covers

WHAT TO SAY

The ZIP includes three contract-level tests. One checks that list_pdfs selects PDF files, another verifies chunk overlap validation, and the third checks that zero-based page metadata is displayed as page one. These tests are useful but limited in scope. They do not test the full Ollama service, embedding model, vector index, or answer accuracy end to end. The source was inspected and Python files were syntax-checked, but this workspace does not include the required LangChain and Ollama environment for a full runtime test.

KEY TAKEAWAY

Contract tests support code quality but do not replace end-to-end evaluation.

Project Limitations

WHAT TO SAY

The current project expects PDFs with selectable text; scanned PDFs require an OCR stage that is not included. Retrieval can bring back semantically related but insufficient passages. Local inference speed depends on computer hardware, and the model may still hallucinate despite temperature zero and system instructions. A classroom demo can demonstrate the architecture, but it cannot establish production-grade reliability. These limitations should be stated clearly when evaluating the application.

KEY TAKEAWAY

A functioning pipeline does not guarantee factual correctness.

Security and Trust Boundaries

WHAT TO SAY

Our code loads a FAISS index using a pickle-based format. Loading a malicious pickle can be dangerous, so the code must only load trusted local index files. Retrieved PDF content is also untrusted and could contain instructions attempting to change the assistant's behavior. The prompt explicitly tells the model to ignore such instructions, but a prompt alone is not a complete security boundary. Finally, PDFs and history logs can contain private information, so file access and sharing matter.

KEY TAKEAWAY

Trusted indexes, untrusted documents, and protected logs are separate concerns.

Future Evaluation: Five Configurations

WHAT TO SAY

The notebook in the uploaded project defines five configurations, A through E, with different chunk sizes, overlaps, and top-k values. This provides a useful starting point for controlled experiments. However, the ZIP does not contain a generated benchmark CSV with measured outcomes. We therefore present the configurations as planned tests, not as completed results. The next step is to run consistent questions for each setting and assess evidence recall, answer faithfulness, and latency.

KEY TAKEAWAY

An experiment plan is not a completed benchmark.

SLIDE 37 / 40 • CONCLUSION • SPEAKER: Sơn Nguyên

Key Takeaways

WHAT TO SAY

We have seen how RAG connects a language model to PDF documents using embeddings and retrieval. Our source code uses multilingual MiniLM, FAISS, a saved-index pipeline, and a LangChain LCEL chain connected to Qwen2.5 through Ollama. Finally, the screenshots show why honest evaluation matters. One refusal behaved as intended, but another plausible answer was not supported by the bundled sample document. The system is a useful learning project, and verification is essential.

KEY TAKEAWAY

Use the code as evidence; verify answers against source documents.

SPEAKER HANDOFF / LIVE DEMO LINE

This concludes the main presentation. Duc and Hai will now demonstrate the working system.